Google Antigravity Skills: Consolidated Guide

1. Introduction to Antigravity Skills

Google Antigravity is an **agentic IDE** designed for an agent-first era. While traditional coding assistants offer autocomplete, Antigravity functions as a "Mission Control" for autonomous agents.

Skills are the modular, file-based extensions that transform the general-purpose agent into a specialist. Instead of relying on a massive, static "System Prompt," Antigravity uses a **progressive disclosure** model: the agent only "equips" the specific logic, instructions, and scripts required for the task at hand.

Key Benefits

- **Reduced Context Saturation:** Prevents "context rot" by loading only relevant instructions.
- **Capability Extension:** Grants agents the ability to execute local scripts (Python, Bash, Node, etc.).
- **Codified Best Practices:** Allows organizations to bake security checks and style guides directly into the agent's workflow.

---

2. The Hierarchy of Customization: Skills vs. Rules vs. Workflows

A critical aspect of mastering Antigravity is understanding when to use each customization tool:

- **Rules (.agent/rules/):** Passive constraints. They are always "on" and act as guardrails (e.g., "Always use PEP 8 style").
- **Workflows (.agent/workflows/):** Active macros. These are user-triggered sequences invoked with a / command (e.g., /review or /deploy).
- **Skills (.agent/skills/):** Cognitive patterns. These are **agent-triggered**. The agent identifies when a skill is relevant to a user's goal and activates it automatically.

---

3. Anatomy of a Skill

A Skill is a directory-based package. While only the definition file is required, a skill can include several optional assets.

File Structure

Plaintext

```
.agent/skills/my-skill/
├── SKILL.md          # Main instructions and triggers (Required)
├── scripts/          # Python, Bash, or Node scripts (Optional)
├── resources/        # Templates, license headers, or JSON schemas (Optional)
└── examples/         # Reference implementations or few-shot prompts (Optional)
```

The SKILL.md File

This uses YAML frontmatter for metadata and Markdown for logic.

- **Name:** A unique identifier for the skill.
- **Description:** The **semantic trigger**. The agent reads this to decide if the skill is relevant.
- **Instructions:** Step-by-step logic the agent must follow.

---

4. Skill Complexity Levels

Skills range from simple prompt engineering to complex "batteries-included" orchestrations:

- **Level 1 (Basic Routing):** Pure prompt engineering (e.g., a Git commit formatter).
- **Level 3 (Scripted Logic):** Involves executing local scripts for deterministic tasks like schema validation.
- **Level 5 (Composition):** Combines scripts, templates (Jinja2), and few-shot examples to scaffold entire tools or workflows.

---

5. Skill Scopes & Storage

Scope	Location	Use Case
Workspace	<project-root>/.agent/skills/	Project-specific scripts or local deployment logic.
Global	~/.gemini/antigravity/skills/	General utilities like JSON formatters or code reviewers.

---

6. Operational Protocol: How the Agent Uses Skills

1. **Discovery:** The agent scans names and descriptions of all available skills.
2. **Activation:** If a match is found, the agent "loads" the full SKILL.md.
3. **Execution:** The agent follows the instructions, potentially running bundled scripts as "black boxes" (using --help to understand parameters).
4. **Release:** Once the task is complete, the context is released to save tokens and maintain performance.

---

7. Best Practices

- **Atomic Design:** One skill per task (e.g., "Review Code" should be separate from "Fix Code").
- **Third-Person Descriptions:** Use clear descriptions like "Generates unit tests using pytest" to help the agent recognize the tool.
- **Safety First:** Use constraints to ensure the agent doesn't execute destructive commands without confirmation.